

IGWT LECTURES

Becoming a Good Teacher

Interactive Graphics and Web Technologies

Handbook for instructors: pedagogy, labs, feedback, integrity, and supervision.

Teaching handbook — how to use these notes

Parent: 06 Becoming a Good Teacher

These notes are a practical handbook for instructors in **Interactive Graphics and Web Technologies (IGWT)**. They assume you already have the curriculum in 01 subjects and the course skeleton in 02 Curriculum Design Advice.

They are not a theory of education degree. They are what a competent colleague would tell you in the week before you teach.

Who this is for

- A new instructor assigned an IGWT course
- An engineer moving from industry into teaching
- A TA who will run labs or live-coding sessions
- An experienced lecturer who has never taught graphics by building
- A program lead who must coach other teachers

How to read

If you have 90 minutes before the semester: 01 What Good Teaching Looks Like, 04 First Day and Syllabus, 19 Checklists and Templates, 23 First Year as an Instructor.

If you have one week: those four, then 02 How Students Learn, 03 Lesson Planning, 06 Live Coding Pedagogy, 08 Feedback and Rubrics, 12 Academic Integrity and AI.

If you have a month: read the folder in numeric order. Do the exercises at the end of each note. Put artifacts in your teaching portfolio (16 Teaching Portfolio and Growth).

If you are in trouble this afternoon: jump.

Situation	Note
Demo will not compile	06 Live Coding Pedagogy
Lab is silent and stuck	07 Labs and Studio
Grading will take 40 hours	08 Feedback and Rubrics
Suspected copying or AI dump	12 Academic Integrity and AI
Student in crisis	11 Mentoring and Office Hours
Argument in class	13 Classroom Difficulties
You have no time	15 Time Management for Instructors

What “good” means here

A good IGWT teacher produces students who can:

1. Explain a concept without the slide.
2. Implement a small version from a blank file.
3. Debug a broken scene, shader, or predicate.
4. Justify a design choice to a skeptic.
5. Finish a project that another machine can run.

If your lectures are admired and those five things do not happen, the teaching is not yet good.

House rules for these notes

- Prefer one concrete classroom move over three slogans.
- Prefer IGWT examples (canvas, WebGL, Three.js, GLSL, glTF, XR) over generic “STEM.”
- Prefer checklists you can print.
- Do not invent student data. When a table is a template, it says so.
- Do not submit student work that was generated from these samples as original research.

Related notes outside this folder

- 01 subjects — what the program contains
- 02 Curriculum Design Advice — weekly course machine
- 04 Computational Geometry — fully worked example course
- Graduation Requirements — exit standards
- 05 Sample Graduation Thesis — what a short thesis looks like

01 — What good teaching looks like

A good teacher is someone whose students can do new things they could not do last month, and who can explain those things under mild pressure.

Charisma helps. Credentials help. Neither is the definition.

Observable signs (walk into the room)

Sign	You will see	Counterfeit
Clarity	Students can restate today's goal in one sentence	Beautiful slides, foggy goal
Practice	Most minutes are students working or watching work being made	Lecture until the bell
Diagnosis	Instructor stops when faces and code diverge	"Any questions? No? Good."
Feedback	Comments name the next change	Scores with no comment
Safety	Students will risk a wrong answer	Only the same three people speak
Standards	Rubric is public before the work	Surprise deductions
Respect for time	Lab starts with a working starter	20 minutes of install pain
Honesty	Instructor says "I do not know; we will find out"	Bluffing through a shader bug

What students remember a year later

They rarely remember your outline. They remember:

- The week they finally saw the hull appear
- The time you sat down and found their `orient` sign error in four minutes
- The critique that was strict and not humiliating
- The project that ran on someone else's laptop
- Whether you learned their name

Design for those memories on purpose.

Three models (use all three)

1. Direct instruction

You explain, you show, they practice a tight task, you check. Best for predicates, matrix conventions, WebGL state, Git, and other things where a wrong model is expensive.

2. Worked-example fading

You show a complete solution while thinking aloud. Next time you leave a hole. Then they do it. Best for shaders, scene graphs, and geometry kernels.

3. Studio / critique

They make something; the room examines it against criteria. Best after they have a minimum technique. See 07 Labs and Studio.

A common IGWT failure is jumping to studio before students have a kernel. Another is staying in lecture after they have a kernel.

The five-pillar week, from the teacher's side

From 02 Curriculum Design Advice:

Pillar	Teacher's job	Failure mode
Theory	One idea, one picture, one invariant	Five ideas, no picture
Demonstration	Make the idea move	Demo of a finished file you cannot reconstruct
Practice	A task with a visible success condition	"Explore Three.js"
Project	A scoped application of several weeks	A second course hidden inside the project
Assessment	Evidence of the learning goal	Memorizing API names

Expertise is a teaching hazard

You have compiled a mental library of defaults: right-handed coordinates, clip space, winding order, `LookAt`, gamma, premultiplied alpha. Students do not have that library. Your job is to **externalize** it.

Before each lecture, write the three things that are "obvious" to you and not to them. Teach those on the board. Examples:

- A vector is not a point; a translation matrix treats them differently
- A fragment shader runs per pixel, not per triangle, unless you say otherwise
- `orient > 0` is a convention you must pick and keep
- A texture that "looks dark" is often a color-space mistake, not a lighting mistake

Authority without theater

Students need you to be competent and fair. They do not need you to be a performer.

- Arrive early. The room should work before they sit.
- Start on time. End on time. That is respect.
- Admit errors in public. Then fix them in public.
- Do not mock tools, prior instructors, or "stupid questions."

- Do not date, flirt with, or privately favor students. See 13 Classroom Difficulties.

A weekly self-check (10 minutes)

After the last lab, write four lines:

1. What could most students do by the end?
2. Where did more than a third get stuck?
3. What will I change in the starter or the explanation?
4. Who needs a check-in before next week?

Keep these notes. They become your teaching portfolio (16 Teaching Portfolio and Growth).

Exercise

Pick one course you will teach. Write five observable outcomes in the form: "By week 6 a student can ___ using ___ without ___." If you cannot fill the blanks, the course is not designed yet.

02 — How students learn

Teaching that ignores how memory and attention work becomes a slide show. This note is the minimum cognitive science an IGWT instructor should use.

Working memory is small

Students can hold only a few new pieces at once. A WebGL first lecture that introduces buffers, shaders, the GL context, clip space, and npm in 70 minutes will overflow almost everyone.

Move: one new idea per 15–20 minutes, then a 3-minute check.

Overloaded lecture	Repaired lecture
"Today: pipeline, VBOs, VAOs, GLSL, uniforms, textures"	"Today: one triangle. The pipeline exists so this triangle can happen."
40 new API names	6 names, written on the board, reused
Shader + React + bundler on day 1	Canvas 2D or a single HTML file first

Cognitive load (three kinds)

1. **Intrinsic** — the idea is actually hard (clip space, degeneracy, PBR). You cannot remove this. You can sequence it.
2. **Extraneous** — the way you present it adds junk (tiny font, unexplained repo, broken starter, three competing diagrams).
3. **Germane** — effort that builds the schema (predict the next pixel, trace one vertex).

Your job is to cut extraneous load so students can spend effort on germane load.

IGWT extraneous-load classics:

- Asking them to install three toolchains in the first lab
- Switching handedness conventions mid-semester without a flag
- Showing a production repo as the first example
- Talking while a shader is compiling and the error is off-screen
- Using a different variable naming scheme than the starter

Dual coding

Geometry and graphics are spatial. Say it **and** draw it. Then make it move.

Topic	Words only (weak)	Word + picture + motion (strong)
Convex hull	"Smallest convex set"	Rubber band animation, then code
MVP matrix	"Model, view, projection"	Move the cube, then the camera, then the frustum
Fragment shader	"Runs per fragment"	Color by <code>gl_FragCoord</code> , then by UV
Sweep line	"Status is the segments the line hits"	Vertical line moving, status listed live

If you cannot draw it, you are not ready to lecture it.

Retrieval beats rereading

Students who only rewatch your recording feel fluent and fail the quiz. Fluency is not knowledge.

Build retrieval into the week:

- Opening: two questions from last week, on paper or a tiny quiz
- Mid-lecture: "Write the sign of `orient` for this picture"
- Lab start: "From memory, list the pipeline stages"
- Homework: one question with no notes, then the rest with notes

Weekly quizzes in 02 Curriculum Design Advice exist for this reason. Do not cancel them because they are annoying to grade. Make them short. See 09 Assessment Design.

Worked examples before open problems

Novices learn procedures from studying complete solutions while attention is guided. Experts underestimate this because they no longer need it.

Sequence for a new algorithm or shader:

1. You implement it, narrating decisions and mistakes.
2. They trace your code on a new input.
3. They change one parameter and predict the picture.
4. They implement a close variant from a starter.
5. Only then: a blank-file or project-sized task.

Skipping to step 5 is how labs become copy-paste sessions.

Desirable difficulty

Struggle that is **in the problem** helps. Struggle that is **in the tooling** rarely helps.

Desirable	Not desirable
Why this point is left of the line	npm peer-dependency hell
Finding the illegal edge	A 400-line starter with no comments
A shader that compiles but looks wrong	A machine that cannot install GPU drivers in the lab

Fix environment pain before class. Put the intellectual pain in the predicate, the invariant, or the visual bug.

Motivation (brief and practical)

Students work when three things are present (self-determination, in classroom language):

- **Competence:** “I can make a triangle / a hull / a material.” Early wins.
- **Autonomy:** constrained choices (“pick the scene, not the stack”).
- **Relatedness:** names, pair work, public demos that are kind.

Grades motivate compliance. Visible progress motivates craft. Use both.

Novice–expert differences in this field

Novice	Expert (you)
Sees a black screen as “WebGL is broken”	Sees a black screen as a checklist (clear color, camera, winding, depth, shader compile)
Copies a matrix from Stack Overflow	Asks which space the vector is in
Treats NaN as random	Treats NaN as a predicate or divide
Wants the library to hide GL	Wants to know what the library hid

Teach the expert’s **checklist**, not only the expert’s **result**.

Transfer does not happen by magic

A student who can convex-hull in the visualizer will not automatically pick a mesh in Three.js. You must assign the transfer.

At the end of a topic, give a 15-minute “where this shows up” slide that is actually a task:

- Hull → silhouette of a 2D gadget in a configurator
- Delaunay → terrain from a height sample
- BVH → click-to-select a part
- Shader normals → a bent normal on a baked asset

Misconceptions you should plan to hit

Keep a running list per course. Examples:

- "Clockwise vs counter-clockwise does not matter if it looks fine on my machine"
- "A texture is just an image; color space is optional"
- "More triangles always look better"
- "`position.set` and a transform matrix are interchangeable thoughts"
- "If it runs at 60 FPS in the lab, it is optimized"
- "AI-generated shader code is a starting point I understand"

Address each with a demo that breaks the belief.

Sleep, spacing, and cramming

A 15-week course already spaces practice if you reuse old kernels. Do that on purpose: Week 7 should still call `orient`. Week 13 should still break on a degenerate mesh.

Say this once, early: the night before the midterm is too late to learn clip space.

Exercise

Take your next lecture. Cut it until a student can do **one** retrieval task at minute 25 and **one** construction task at minute 55. If you cannot, the lecture has too many ideas.

03 — Lesson planning

A lesson plan is not a script of everything you will say. It is a sequence of student actions with times, materials, and a fallback.

If you only have slides, you have a talk. If you have this sheet, you have a class.

The one-page plan

Copy this for every meeting. A filled example is at the end.

```
Course / week:
Learning goal (one sentence, observable):
Success check (what I will look at):
Materials (starter, slides, demo repo, quiz):
0-10   Hook / retrieval
10-25  Idea 1 + picture
25-40  Live demo / live coding
40-50  Student attempt (tiny)
50-65  Idea 2 or common bugs
65-75  Recap + next lab preview
If the demo dies:
If we run long, cut:
If we run short, add:
Students who will need a check:
```

Seventy-five minutes is the default lecture in 04 Computational Geometry. Adjust the blocks; do not drop the success check.

Start from the end

Write the learning goal as a verb a stranger could grade.

Weak	Strong
Understand transformations	Apply a translation then a rotation to a point on paper and in code
Learn shaders	Write a fragment shader that colors by UV and explain the two compile errors I will plant
Know convex hulls	Trace Andrew's chain on 8 points and name the left-turn test

Then write the **evidence**: a sketch, a 10-line function, a screenshot, a two-sentence explanation.

If you cannot name the evidence, you are planning a vibe.

Backward design (course scale)

1. What must they do in the final project or exam?
2. What weekly performances build that?
3. What lecture/lab pair produces each performance?

Do not start from “Chapter 4 of the textbook” unless that chapter is the performance.

For IGWT, the final performances are usually: a running scene, a short report, a demo, and answers to “what happens when this input is degenerate / this GPU is slower / this asset is 80 MB.”

Timing is a moral issue

If you plan 90 minutes of content for 75 minutes, you will skip the student attempt. That is the part that causes learning. Cut content, not practice.

Rule: if a block has no student action, it may not exceed 15 minutes.

Student actions include: write, draw, predict, type, vote, pair-explain, run a test. Nodding is not an action.

Prepare materials before the week begins

From 02 Curriculum Design Advice, the professor prepares slides, live-coding examples, lab manual, homework, project spec, recordings, and a repo. For one week that means:

Artifact	Done when
Starter compiles on a lab machine and on a student laptop	Sunday
Solution exists, but is not in the student branch	Sunday
Lab manual has expected output (screenshot)	Sunday
Quiz has 5 items and a 3-minute key	Sunday
Demo file opens in one command	Before you sleep the night before
Backup: recorded 4-minute version of the demo	Before you sleep

The most common amateur move is writing the lab at 8:00 for a 9:00 class.

Board plan

Decide what will be on the board at the end. Students photograph the board. If the board is a mess of half-erased matrices, that becomes the notes.

Leave a reserved strip for:

- Today's goal
- Named conventions (`orient > 0` means left)

- The pipeline or invariant of the day

Differentiation without three courses

You will have students who finished a game in high school and students who still fight Git.

Plan **one core path** and **two valves**:

- **Support valve:** a hint sheet, a failing test with a message, a 10-minute TA table.
- **Challenge valve:** "add a second light," "handle collinear points," "measure $n=10k$."

Do not make the challenge required for a full score unless it was in the spec.

Pair and group planning

Pairs in lab: similar-ish skill, or a rotating "driver/navigator" so the strong student cannot monopolize the keyboard. See 07 Labs and Studio.

Never assume they know how to pair. Teach the roles in week 1.

The 10-minute rehearsal

The night before, stand up and:

1. Open the starter from a clean clone.
2. Run the demo.
3. Say the first two minutes out loud.
4. Write the one question you will ask at minute 20.

If the clone fails, the lesson is not ready.

After-class note (same day)

Three bullets: what landed, what broke, what to change in the starter. This is how a course becomes good in year 2.

Filled example (Computational Geometry, Week 2)

Course / week: Comp Geom / Week 2 Geometric primitives

Learning goal: Students can implement `orient(a,b,c)` and interpret its sign.

Success check: 8/10 lab pairs pass the hidden collinear test.

Materials: visualizer starter, 5-question quiz, board diagram of three points.

0-10 Last week: predicate vs construction. Two pictures.

10-25 Cross product in 2D. Board. Three numerical examples.

25-40 Live-code `orient`. Plant a sign error. Fix it.

40-50 Students compute two triples on paper, then in the starter.

50-65 Collinear, duplicate points, “almost zero.” EPS policy.

65-75 Recap. Lab: draw the sign as color. Homework: 3 written + implement `onSegment`.

If the demo dies: use the recorded 4-minute clip; board the code.

If we run long, cut: the third numerical example.

If we run short, add: “where this is used in picking.”

Check: anyone who missed week 1 lab.

Exercise

Write next week’s one-page plan. If it does not fit on one page, you are writing a chapter, not a lesson.

04 — First day and syllabus

The first meeting teaches students how this course will feel. They decide, in the first 20 minutes, whether effort will be rewarded and whether confusion is allowed.

Do not spend the first day reading the syllabus aloud.

Goals of day one

1. They can name the course goal in one sentence.
2. They have produced one small visible thing.
3. They know how to get help and how they will be graded.
4. You have seen who is already lost on install.
5. They have heard the integrity and AI rules in human language, not only in legal text.

A 75-minute first-day script

Minutes	Do this	Do not do this
0–8	Who you are, why this course exists, one picture of the end project	Your full CV
8–18	The contract: time, tools, grading, late work, AI	Reading 8 pages of policy
18–28	Names and a 60-second pair intro: “what I want to make”	Icebreakers unrelated to the work
28–55	A complete tiny win (see below)	Tooling lecture with no output
55–70	Clone the repo, run the starter, fix the one machine that fails	“Install this at home, good luck”
70–75	What to do before next time (one page, one task)	A 40-item reading list

Tiny wins that fit day one

Pick one. Finish it.

Course	Tiny win
Intro programming	A page that prints their name and a random number
Web technologies	A heading, a color, inspect it in DevTools
Computer Graphics I	A cube they can rotate with the mouse
Computational geometry	Three points and a colored <code>orient</code>
WebGL	One triangle in a single HTML file
Three.js	A glTF they did not model, lit, rotating
Shaders	A full-screen gradient they can edit live
Capstone	A repo, a board, a 4-week milestone on paper

The point is competence, not coverage.

Syllabus: what must be written

Put this in a document they can find at 11 pm. Also put a 1-page version in the repo README.

Required blocks:

1. Course goal and weekly rhythm (lecture / live coding / lab / homework / quiz)
2. Prerequisites you will actually enforce
3. Software and hardware (and the lab alternative if their laptop cannot)
4. Grade table with weights
5. Late policy (be specific: hours, caps, what "emergency" means)
6. Collaboration policy: what they may share, what they may not
7. AI policy: allowed uses, required disclosure, forbidden uses — see 12 Academic Integrity and AI
8. Accessibility and how to request accommodation — see 10 Inclusive Teaching and Accessibility
9. Communication: where you will answer (LMS / email / office hours), and your response-time promise
10. Schedule with topics, not just week numbers
11. How to get help before they fail a week
12. What "done" means for the final project

Tone: firm and human. "Late work loses 10% per 24 hours, max 50%, except documented emergencies" is better than "no late work" if you will make exceptions anyway. Do not write a rule you will not keep.

The verbal contract (say this)

You will be confused in this course. That is normal. The unacceptable thing is staying confused alone for two weeks. Come to lab, office hours, or the forum with a screenshot and what you already tried.

I will not surprise you with hidden grading criteria. I will surprise you with degenerate inputs.

I will not pretend AI does not exist. I will require you to be able to explain and modify anything you submit.

Names

Learn names faster than you think you need to.

- Photo roster if the university provides one
- Name tents for two weeks in smaller rooms
- Call the roster when you return work
- In lab, use names when you sit down: "Maya, show me the test that fails"

Students work harder for a person who knows they exist.

Diagnostic, not a gate

Day one or week one homework: a short, ungraded or low-stakes diagnostic.

- Can they write a loop and a function?
- Can they clone and run?
- Can they add two vectors?

Use it to assign support, not to shame. If half the room fails vectors, your Week 2 graphics lecture must change. See 02 How Students Learn.

Safety and power

On day one, state:

- How to report harassment
- That you will not meet a student alone in a closed room after hours without a public note (use a booked office, door open, or a TA present)
- That grades are not negotiated by charm

You are not their friend. You can be warm and still be their examiner.

Common first-day failures

- Death by syllabus

- Install session that eats the tiny win
- Promising a Discord you will not moderate
- Joking that “half of you will drop” (some will believe you)
- Over-selling how easy shaders are
- Under-selling the weekly hours

First-week email (send after day one)

Five lines:

1. Link to syllabus and repo
2. The one task due before meeting 2
3. Office hours
4. “If the starter does not run, come to X with your laptop”
5. The AI/collaboration sentence

Exercise

Write the 1-page syllabus. Time yourself. If you cannot do it in 40 minutes, the course policies are not decided.

05 — Lecture craft

A lecture in this program is a guided encounter with one idea, not a transfer of slides. Students can read. They cannot, by themselves, watch an expert decide what to ignore.

One idea

If a colleague asks “what was lecture 6 about?” you should answer with a noun phrase: *clip space*, *sweep line status*, *Cook–Torrance at a glance*, *the scene graph vs the scene*.

If your answer is a list of six topics, it was not a lecture. It was a table of contents.

The shape of 75 minutes

Phase	Time	Purpose
Retrieve	5–8 min	Last week’s idea, under a little pressure
Frame	3 min	Today’s question and why a graphics person cares
Build	20–25 min	Board + pictures + one wrong turn
Show	15–20 min	Live demo or live coding
Attempt	8–12 min	They try a fragment
Land	5 min	Recap, the invariant, the lab hook

Times flex. The attempt does not disappear.

Openings that work

- A broken picture: “Why is this mesh black?”
- A lie students believe: “More triangles will fix this silhouette.”
- A 30-second silent demo, then “write what you think happened.”
- A single numerical surprise: three collinear points and a naive `if (area == 0)`.

Openings that fail: “Last time we talked about... and today we will continue...”

Board over slides

Slides are for photographs, code you should not hand-write, and animations. The argument belongs on the board.

Write large. The back row is part of the class.

Do not talk to the board. Say the sentence, then write the short form.

Do not erase the invariant. Park it.

For matrices, write the **spaces** (`object` → `world` → `view` → `clip`) more carefully than the numbers. Students lose the spaces first.

Voice and body

- Stand where they can see the demo machine and the board.
- Face the room when you ask a question. Wait. Waiting is the skill. Count to seven in your head.
- Walk the aisles during the attempt. You are collecting data.
- Do not pace like a talk show. It is visually noisy.

If you are soft-spoken, use a microphone. Pride is not a pedagogy.

Language

Prefer short sentences. Define a term the first time, then use it consistently.

Say	Avoid
"The camera looks down –Z in this convention"	"Obviously the camera..."
"This function returns a sign"	"We just do the cross product thing"
"I am going to make a mistake on purpose"	Silent magic edits

Avoid filling pauses with "right?" and "make sense?" Those are not checks. Real checks are in 20 Questioning and Discussion.

Slides, if you use them

- One idea per slide
- Diagrams, not paragraphs (see 02 Curriculum Design Advice)
- Code no smaller than about 20pt in the room
- Dark-on-light or light-on-dark, not gray-on-teal
- Alt-text or a spoken description for every important figure (10 Inclusive Teaching and Accessibility)
- Put the slide deck in the repo **after** class if you want them listening; before class if you want them annotating. Pick one and keep it.

Never read a bullet list that they can read faster than you can speak.

Handling the middle slump (minute 35)

Attention drops. Change the channel:

- Switch from slides to board
- Switch from talk to typed code

- A 90-second pair talk: “explain the last diagram”
- A prediction: “what color is the pixel at the center?”

Endings

The last three minutes are not “any questions.” They are:

1. The invariant or pipeline, said again
2. The mistake they are about to make in lab
3. The exact homework prompt

Then stop. Lingering while they pack trains them to ignore endings.

Recording

Record if you can. A recording is not a substitute for attendance if the course is studio-heavy; say that.

When you record:

- Mic on you, not the air conditioner
- Zoom the IDE to 140%
- Narrate clicks (“I am opening the fragment shader, not the vertex shader”)
- After a live fix, leave a 20-second summary for later viewers

Guest lectures

A guest is a gift and a risk. Brief them: audience level, 15 minutes of demo, 10 minutes of Q&A, no confidential employer code. You stay in the room. You connect the guest to next week’s lab.

Exercise

Take a slide deck you already have. Delete half the slides. Add one student attempt. If the lecture dies, the deleted slides were the lecture, and that is the problem.

06 — Live coding pedagogy

Live coding is the signature move of this program. Students learn by watching an expert think, stall, read an error, and recover. A polished video of a finished file teaches the result. Live coding teaches the work.

If you only ever paste finished solutions, you are hiding the job.

Why it works

- It makes process visible: where to look first, what to ignore.
- It sets a humane pace. You cannot type as fast as a recording.
- It licenses mistakes. If you can be wrong in public, they can be wrong in lab.
- It creates a shared object. After class, “the cube from minute 40” is a real cube.

Rules

1. **Start from the same starter they have.** No secret sauce in a private folder.
2. **Zoom the editor.** 140–160%. Theme with contrast. Ligatures optional; wrapping not optional.
3. **One file on screen** unless you are teaching imports.
4. **Narrate decisions, not keystrokes.** “I need a vec3 in world space” is better than “now I type u_model.”
5. **Make a mistake on purpose** if the session is going too clean. Then show the checklist.
6. **Stop and let them type** after 8–12 minutes. Live coding is not a movie.
7. **Never live-install.** The environment is ready or you abort to the backup.

A 60-minute live-coding arc

This matches the “Live Coding / Demo” block in 02 Curriculum Design Advice.

Minutes	Move
0–5	Recap the goal. Open the starter. Run it so they see the “before.”
5–20	Implement the first slice. Talk. Type slower than you want.
20–25	They reproduce that slice, or predict the next picture.
25–40	Second slice. Include one error (compile or visual).
40–50	They try the error-fix checklist on a planted bug in the starter.
50–60	Clean up, extract a function, write the one comment that matters, commit.

Ending on a commit teaches that work is not done when the picture appears.

What to narrate

Always:

- What space a vector is in
- What you are about to compile or refresh
- How you know it worked (screenshot, test, pixel color)
- The first place you will look if it is black / NaN / inverted

Sometimes:

- Why you rejected an API
- Why you are not optimizing yet
- Why this is a predicate, not a construction

Never:

- A two-minute rant about a framework
- Silent scrolling through files
- "I'll just copy this from my other repo"

The black-screen checklist (teach it, post it)

When the scene is black or blank:

1. Is the canvas in the DOM and sized?
2. Are we clearing to a color you would notice (not black-on-black)?
3. Did the shader compile? Read the info log.
4. Is the camera looking at the object?
5. Is the object in front of the near plane?
6. Winding / culling?
7. Depth test / write?
8. Are attributes bound to the right locations?
9. Is the texture loaded yet (async)?
10. Is the color space or alpha making it "invisible"?

For computational geometry, the sibling list is: sign convention, EPS, input order, duplicate points, and "did you sort a copy?"

Print this. Students should run it before they raise a hand.

Failure is part of the lesson

When the unexpected happens:

1. Do not apologize for 90 seconds. Breathe.
2. Say "this is the real job."
3. Use the checklist out loud.
4. If 5 minutes pass, switch to the backup recording or a known-good commit.

5. After class, write the incident into next year's notes.

Students remember how you treat a broken demo more than they remember a perfect one.

Pair live coding

Two instructors (or instructor + TA) can work as driver/navigator. The navigator faces the room and translates. This is excellent and must be rehearsed once.

Do not let the navigator become a second monologue.

Speed

You are too fast. Almost every expert is.

- Type the identifier, then pause.
- After an error, wait so they can read it.
- Do not use 15 IDE shortcuts they do not have.

If advanced students are bored, give them the challenge valve on paper. Do not speed up past the middle of the room.

Accessibility

- Speak what you type if anyone is using a screen reader or is far away.
- Do not rely on red squiggles alone.
- Provide the final file after class, plus a short "decisions" note, not only a dump of keystrokes.

See 10 Inclusive Teaching and Accessibility.

Anti-patterns

- Live-coding a 400-line engine on day one
- Switching windows every 10 seconds
- Dark gray text on black, 12px
- "I'll clean this up later" and never extracting the function
- Finishing the feature after they have packed up, so only the recording sees the point

Exercise

Record yourself live-coding for 12 minutes. Watch at 1.5x. Note every unexplained jump. Those jumps are where the class gets lost.

07 — Labs and studio

Labs are where the course happens. If lecture is the map, lab is the terrain. A bad lab is a silent room of stuck people and one instructor who answers the same Git question twelve times.

Studio (project weeks, critiques) is a lab with public criteria and less new content. Week 14 of computational geometry is a model: Week 14 Project Studio.

Before the door opens

- Starter cloned and run on a lab image **and** on a typical student Windows/Mac laptop
- Lab manual: goal, time box, expected screenshot, hints, common mistakes
- Tests or a visual oracle so they know "done"
- A parking lot on the board for issues that are not today's goal
- TAs briefed on the one misconception of the week

If install is the lab, you did not have a lab. You had an IT session. Put install in a pre-lab video and a help hour.

Opening the lab (10 minutes, not 30)

1. Today's success condition (one screenshot or one test)
2. The mistake you expect
3. Where the hint sheet is
4. Pair roles if pairing
5. "Try for 8 minutes before you call me, unless the starter does not run"

Then stop talking. Your talking is their most expensive resource.

While they work

You circulate. You do not sit at the front answering email.

The 4-minute desk visit:

1. "Show me what it does now."
2. "What did you already try?"
3. Look at the screen, not only their story.
4. Give **one** next action, not a lecture.
5. Leave. If you stay, you will finish it for them.

If three groups have the same bug, stop the room. Teach that bug once. That is diagnosis, not interruption.

Pairing

Default in early courses: pairs. Default in later courses: optional pairs plus individual kernel tests so one person cannot hide.

Driver / navigator (25-minute swaps):

- Driver types
- Navigator holds the checklist and the spec
- Both must be able to explain the last commit

If one student always drives, you will graduate a typist and a spectator. Swap on a timer you can hear.

Help-seeking that you will accept

Train this script in week 1:

```
I expected: (screenshot or test name)
I got: (screenshot or error)
I tried: (two things)
I think: (one hypothesis)
```

Refuse, kindly, to debug from “it doesn’t work.” You are teaching them to be colleagues.

Studio rules (critique)

A critique is not a roast and not a compliment circle.

Before: criteria on a slide. “We are looking at silhouette, frame time, and whether the README runs.”

During:

- Maker speaks for 60–90 seconds: goal, what is broken, what they want advice on
- Room speaks to the **work**, using the criteria
- Instructor models one high-quality comment
- Timebox. A 12-person studio cannot give everyone 15 minutes

After: each team writes one change they will make this week. You collect that line. That is the feedback loop.

Forbidden comments: “I just don’t like it.” Required shape: “The shadow acne is visible on the floor; a depth bias or closer near-plane would address criterion 2.”

When the room is too quiet

Quiet can mean thinking or drowning.

- Ask two people to project their screen (volunteers first)
- A 5-minute “gallery walk”: stand, look at two neighbors, return
- A mid-lab poll: “Who has a triangle / a hull / a compile?” Hands. Then you know.

If nobody has a triangle at minute 40, the starter or the lecture failed. Do not blame “this cohort.”

When the room is too loud

One conversation at a time during a stop-the-room moment. During work, noise is often healthy. If two tables are socializing off-task, stand near them and ask to see the screenshot.

End of lab

Last 7 minutes:

- Commit and push (teach this until it is a reflex)
- One sentence in the lab log: “works / broken / blocked on X”
- Preview of homework, which should continue the same object, not a new universe

Do not introduce a new topic in the last 7 minutes.

Lab manual skeleton

```
Title and week
Time: 2–3 hours
Goal (observable)
Starter: how to run
Task 1 (must)
Task 2 (must)
Task 3 (should)
Stretch (may)
Expected output (image)
Common mistakes
How to submit
How we will grade (link to rubric)
```

Exercise

Write this week’s lab so that a TA who was not in lecture can run it. If they cannot, students cannot.

08 — Feedback and rubrics

Feedback is information a student can use to change the next version of the work. A number is not feedback. A paragraph that restates the spec is barely feedback.

If they cannot act before the next grade, you wrote a post-mortem, not a teaching move.

Speed vs quality

In a 40-person course you will not write an essay on every lab. You must choose a system.

Method	Use when	Cost
Rubric + 2 specific comments	Weekly labs	Medium
Autograder + one human comment on the visual	Kernels, predicates, unit-testable code	Low after setup
Audio note (2 minutes)	Projects, critiques	Low time, high warmth
In-lab oral feedback, recorded on a checklist	Studio weeks	Lowest writing
Full written review	Thesis chapters, final reports	High; schedule it

Never grade only by “I know quality when I see it.” That is how bias and exhaustion leak.

Rubrics students see in advance

Publish the rubric with the assignment. If a criterion is not on it, you may praise it, but do not punish its absence.

Analytic rubric (preferred for code + report):

Criterion	Below	Meets	Exceeds
Runs	Does not run from README	Runs on a clean clone	Runs + records a known limitation
Kernel / technique	Missing or wrong idea	Correct on typical input	Degenerate case handled or documented
Visual / UX	No way to see the idea	Idea is visible	Controls or overlay teach the idea
Code hygiene	Unreadable, secrets, dump	Named, structured, no secrets	Tests or comments at decisions
Write-up	Story without decisions	Decisions and one figure	Limitations and a next experiment

Holistic rubric (letter-level descriptions) is fine for a small studio with experienced graders. It is a fight in a large course.

Comments that change work

Use a consistent shape:

1. **Name the evidence.** "Your hull skips the middle point on the upper chain in test 04."
2. **Name the principle.** "The left-turn test must use the last two hull points, not the first."
3. **Name the next action.** "Draw the chain for that fixture before you change code."

Avoid:

- "Messy"
- "Good job!" as the only line
- Rewriting their program in the comment box
- Sarcasm

If you are angry, wait 12 hours. Grades written at 1 a.m. are meaner than you think.

How much to fix

You are not their pair programmer in the gradebook.

- Mark the first instance of a pattern ("winding is inconsistent in three files; fix the convention in the kernel")
- Do not line-edit every file
- For writing: mark one page densely as a model, then require them to apply it

Consistency across TAs

- Grade one sample together before the pile
- Use a shared comment bank for the five common bugs
- Blind the names if the LMS allows it, especially on reports
- Instructor re-grades a 10% sample

If two TAs differ by a full letter on the same project, the rubric is not operational yet.

Returning work

Return labs before the next related lab. A Week 3 grade arriving in Week 7 trains them to ignore comments.

Spend 5 minutes in the next lecture on the **class-wide** pattern: "Half of you inverted the near-plane. Here is the picture." That is efficient feedback.

Regrade policy

Write it once:

- Regrade requests in 5 days, in writing, pointing to a rubric cell

- The whole submission may be re-read (score can go down)
- "I need a higher GPA" is not a request

This protects you and them.

Praise

Praise **process and decision**, not identity.

- "You isolated the shader compile log before changing the mesh. That is the right order."
- Not: "You are a natural at graphics."

Identity praise makes the next failure feel like a verdict.

Feedback on live performance

For presentations and defenses, use a card they can photograph:

Item	Notes
Problem in one sentence	
Demo reliability	
Technical depth of answers	
Visuals / timing	
One improvement for next time	

See Graduation Requirements and 22 Supervising Theses.

Exercise

Take last year's assignment. Write a 4-row rubric that a new TA could apply. Then grade three old submissions. If you disagree with yourself, the rubric is still vague.

09 — Assessment design

Assessment is how you find out whether the course worked. It is also how students decide what to study. If you test API trivia, they will memorize API trivia. If you test invariants, degeneracy, and a running system, they will practice those.

Align to goals

For each learning goal, name one piece of evidence.

Goal	Poor evidence	Better evidence
Understand clip space	Define clip space	Transform a point on paper; explain a clipped triangle
Implement hull	"Write a hull" with no tests	Hidden fixtures + visual
Optimize a scene	"Discuss optimization"	Before/after frame time and what they changed
Communicate	Long report nobody reads	6–8 pages + a 5-minute demo

If a goal has no evidence, it is decoration. Delete it from the syllabus or add a task.

The program's default mix

From 02 Curriculum Design Advice and 04 Computational Geometry:

Component	Role	Notes
Labs	Can they do this week's skill?	Frequent, lower stakes
Homework	Can they do it alone, with time?	Mix of code and short writing
Weekly quiz	Retrieval	10–15 minutes, start of lecture
Midterm	Can they explain without a laptop?	Pictures, signs, complexity, one trace
Project	Can they combine skills?	Demo + report, not only code
Participation	Did they show up as a colleague?	If you grade it, define it or do not grade it

Do not add a huge written final on top of a huge project unless the department forces you. Students have finite hours. See 15 Time Management for Instructors.

Quizzes that are worth it

Five items. Ten minutes. Same week as the idea.

Good items:

- A picture: mark the left turns
- One complexity: "Andrew's chain after sort is ___"

- One “what is wrong with this shader error”
- One convention: handedness, winding, color space
- One short “what would you try first if the scene is black?”

Bad items:

- Name four Three.js classes
- A 20-minute proof
- A trick question about an API flag you mentioned once

Grade them the same day if you can. The point is retrieval, not a second midterm.

Written exams in a building course

You still need some paper. Keyboards hide whether they understand.

Exam design:

- A figure on every other question
- One trace of an algorithm they have seen
- One “here is a wrong explanation; fix it”
- One transfer: “where would you use this in picking?”
- No surprise topic that never appeared in lab

Provide a formula sheet if the goal is not memorizing the matrix. Say so in week 1.

Projects

A project spec needs:

- A **must** list (the grade of “meets”)
- A **should** list
- A **may** list (exceeds)
- Forbidden scope (“do not also write a physics engine”)
- Deliverables: repo, README, 60-second video or in-class demo, short report
- Rubric
- Milestone dates, not only a final date
- Academic integrity and AI rules

Milestones save lives:

Week	Must show
2	Repo + running starter + chosen topic
4	Core algorithm or scene with one interaction
6	Degeneracy or performance note + draft README
8	Feature freeze + report draft
9	Demo

If you only have a final deadline, you will meet the project for the first time when it is too late to teach.

Group work

Group projects need individual accountability:

- A kernel or scene file each person owns
- A short individual quiz or oral on the shared system
- A contribution statement you actually read
- A way to fire a non-contributor with a documented process, used rarely and in writing

Otherwise you are grading the strongest student's weekend.

Validity, reliability, fairness (in plain language)

- **Valid:** it measures the goal (a pretty demo is not a hull)
- **Reliable:** two TAs would agree
- **Fair:** time, language, and hardware do not secretly dominate the grade

Fairness examples:

- Provide a lab machine option for heavy GPU work
- Do not require a \$3,000 laptop
- Extra time as formal accommodation, not as a secret deal
- Language: grade graphics English for clarity, not for literature

Grade distributions

Do not curve to punish a strong year. Do not inflate to hide a weak exam.

If the median is a D, the assessment or the teaching failed. Investigate before you "adjust." If the median is an A and the projects do not run, the rubric failed.

Exercise

Write a 10-minute quiz for your next lecture **before** you write the slides. Then write only the slides that make that quiz fair.

10 — Inclusive teaching and accessibility

Inclusion is not a poster. It is whether a student with a different body, language, money, or background can do the same course without heroic effort.

Graphics courses accidentally exclude people: tiny code, flashing shaders, assumed gaming culture, expensive laptops, “jokes” about who is bad at math.

Universal design (do these for everyone)

- High-contrast slides and IDE
- Large live-code font
- Captions on recordings
- Materials before class when possible (especially readings and starter)
- Multiple ways to participate: speak, write, type in a shared doc
- Assignments that do not require hearing color names alone (“the red cube”) — also say position and name
- A lab machine path for students whose laptops cannot run WebGL well
- Text descriptions of key diagrams in the notes

These help more people than the one student who requested them.

Legal accommodations

If the disability office sends a letter (extra time, note-taker, rest breaks, flexible attendance), you implement it. You do not diagnose. You do not bargain it down because “industry will not give extra time.” Industry is not your exam hall.

Keep a private checklist. Never announce an accommodation to the class.

Vision, hearing, motor

Barrier	Classroom move
Low vision	160% editor, verbalize what you point at, provide files
Color vision	Never encode meaning in color only; use shape, label, dash
Hearing	Mic, captions, do not talk facing the board only
Motor / fatigue	Extra time, less live-typing pressure, allow speech-to-text
Photosensitivity	Warn before flashing shaders; provide a still alternative

Seizures and migraine are not theoretical in a shader course. Warn.

Language

Many IGWT students are not native speakers of the instruction language.

- Slow down technical nouns; write them
- Allow a glossary
- Grade code and diagrams more than idiomatic essays, unless writing is the goal
- Do not mock accents or “broken English”
- Pairing: do not always put the weakest speaker with the strongest coder as unpaid translator

Money and equipment

Do not require a personal VR headset in week 2. Provide a checkout pool or a shared lab hour.

Do not require paid software when Blender, a browser, and a text editor will do.

Be careful with “buy this course on Udemy” as unofficial homework.

Culture and belonging

- Use a variety of names and examples (not only sci-fi guns and anime)
- Do not assume everyone played games or knows Twitch
- Interrupt jokes about gender, nationality, or “graphics bros”
- Call on a range of people; do not let one student become the room
- Show work by a range of practitioners in slide credits

Belonging is not a pizza party. It is whether a student can ask a basic question in week 3.

Neurodiversity

Some students cannot take a surprise oral. Some cannot sit a 3-hour lab without a break. Some hyperfocus and skip the report.

- Publish agendas
- Avoid sudden changes of room or stack
- Allow noise-cancelling headphones in lab if they can still hear a stop-the-room call
- Written instructions, not only spoken

Religion, fasting, family

Be careful with evening demos in the last week of a fasting month, and with Sunday deadlines if your population cannot use that day. A Monday 09:00 deadline is often kinder than Sunday 23:59.

Content warnings (rare but real)

Medical visualization, some anatomy assets, and violent game references can be hard. Offer an alternate asset when the learning goal is the pipeline, not the picture.

What inclusion is not

- Lowering the standard of the kernel
- Doing the project for a student
- Hiding feedback to be "nice"

High standards and high support. That is the pair.

Exercise

Pick one lecture. Add captions, enlarge the code, replace one color-only diagram, and write a text alternative for the main figure. Time it. That is the real cost; budget it.

11 — Mentoring and office hours

Office hours are not a second lecture for the students who like you. They are a clinic. Mentoring is the longer relationship: projects, theses, careers, and the student who is about to disappear.

How to run office hours

Publish them. Same place, same time, most weeks. If you cancel, say so early.

Format options:

Format	Best for	Watch-out
Open room, queue on the board	Labs, install, short bugs	One student monopolizes
15-minute bookings	Thesis, grades, sensitive topics	Empty slots if you over-book friction
Hybrid: first hour drop-in, second hour booked	Mixed courses	You must enforce the split
TA hours + your hours	Large courses	Students must know who is for what

In the room:

- Door open if you are alone with one student
- Laptop facing so you both see
- Timer visible for the queue
- “Show me the repo,” not “tell me the story for 10 minutes”

The 12-minute bug clinic:

1. Reproduce
2. Hypothesis
3. One experiment
4. They type the fix
5. They say what they will do next

If you type the fix, you feel useful and they learned your keyboard.

Who does not come

The students who need you most often do not appear. They are ashamed, working a job, or lost enough that they cannot form a question.

Moves:

- After a failed quiz, write a short note: “Come Thursday with the starter; no speech required.”
- Stay 10 minutes after lab

- Watch who never submits; email once, early, with a concrete offer
- Do not rely only on “my door is always open.” That sentence is not a system

Academic mentoring

Help them choose a project that is **narrow enough to finish**. The amateur mentor inflates scope because the idea is exciting.

A good undergraduate project: one core algorithm or one rendering feature, one evaluation, one honest limitation. See 22 Supervising Theses and Graduation Requirements.

Ask every two weeks:

- What runs?
- What is the risk?
- What will you cut?

Career mentoring (keep it in your lane)

You can:

- Review a portfolio site for clarity and honesty
- Explain what a graphics/web internship looks at (repo, demo, README)
- Introduce them to a guest or alum **with permission**
- Write a letter if you have seen their work

You should not:

- Promise a job
- Push them into your startup as unpaid labor
- Write a letter for someone whose work you have not seen
- Give immigration or legal advice

Emotional distress

You are not their therapist. You are a mandatory reporter in some systems; know your campus rule.

If a student is in crisis:

- Listen, do not interrogate
- Do not promise total secrecy if you cannot keep it
- Walk them to counseling services if they agree, or give the number and stay until they have a next step
- If there is imminent risk of harm, contact campus emergency services
- Afterward, write the facts (time, what was said, what you did), not a novel
- Take care of yourself; tell a chair if you need backup

Do not diagnose. Do not tell them to “just push through the shader.”

Power, boundaries, dual relationships

- No dating students in your course or under your supervision
- No drinking with undergraduates you grade
- No private money lending
- Gifts: follow campus policy; when in doubt, refuse kindly
- Social media: do not follow student private accounts; keep a public professional page if you want

Warmth is allowed. Confusion about roles is not.

Letters of recommendation

Say yes only if you can be specific. Ask for:

- Deadline and form
- CV and unofficial transcript
- Project they want you to mention
- Draft of their statement if they have one

Write about work you saw. "Hard-working and passionate" is a discarded letter.

If you cannot write a strong letter, say so early: "I can confirm you took the course; I cannot write a competitive letter because I did not see project-level work."

Mentoring strong students

Do not only spend time on firefighting. Give strong students a harder valve, a paper, a TA tryout, or a tighter critique. Neglecting them is also bad teaching.

Exercise

Write your office-hours script on a card: hours, queue rule, 12-minute clinic, emergency contacts. Put the card on the door.

12 — Academic integrity and AI

Students will use other people's code and generative models. Pretending otherwise is not a policy. Your job is to define **allowed collaboration**, **required disclosure**, and **what mastery still means**.

This note is about teaching and assessment design, not about evading detection.

Say the rule in week 1, in human language

A usable policy has four lines:

1. **You may** discuss ideas, draw pictures, and compare error messages.
2. **You may not** copy another student's kernel, shader, or report, or share your repo with a student who has not submitted.
3. **You may use AI** as a rubber duck, for syntax, and for first drafts of comments **if** you disclose the tool and you can explain every line you submit.
4. **You may not** submit an AI-generated project you cannot modify live.

Put the same four lines in the syllabus, the repo, and the project spec. See 04 First Day and Syllabus.

Design so cheating is less attractive

People copy when the work is impossible, unclear, or irrelevant, or when they are drowning.

- Starters that run
- Rubrics that are public
- Milestones
- Help that arrives before the night before
- Assessments that require **process** (trace, oral, change request)

If the only artifact is a zip of source, you are grading the internet.

What to assess that AI is bad at (today)

- Changing a running system under a new constraint ("handle collinear points," "cut texture memory by half")
- Explaining a picture you point at
- A 10-minute oral: "what does this function return on this triple?"
- A paper trace of an algorithm
- A live tweak in office hours or demo day
- Git history that matches the story (not 1 commit at 3:59 a.m. with 4,000 lines)

You are not trying to "catch AI." You are trying to see a mind.

Disclosure template (give them this)

Tools used: (none / Copilot / ChatGPT / other)
I used them for: (syntax / idea / draft paragraph / debug suggestion)
I did not use them for: (the kernel / the report argument / ...)
I can explain: (file names)

A disclosure is not a confession of guilt. It is part of professional practice.

Collaboration vs collusion

Allowed	Not allowed
Whiteboard of the sweep-line idea	Photos of another student's finished code
Pair lab when the spec says pair	Pair homework when the spec says individual
Using Three.js docs and examples	Pasting a whole example and changing variable names
Stack Overflow for an error you still fix	Paying someone to finish the project

When in doubt, they should ask **before** the deadline.

Process when you suspect a problem

Stay calm. You are an investigator of the work, not a prosecutor of a soul.

1. Compare artifacts (structure, comments, identical bugs, identical wrong comments).
2. Invite the student to a meeting: "Walk me through this function."
3. Take notes. Another faculty member present if policy says so.
4. Follow the **department process**. Do not invent a private punishment.
5. Do not post the case on Slack. Do not gossip.

If they can modify and explain, the learning goal may still be met; you may still mark a policy violation if they lied about authorship. If they cannot explain, they have not met the goal.

Never threaten. Never promise "this stays between us" if the university requires a report.

AI-specific classroom moves

- Live-code with AI **once**, as a demo: show a plausible wrong shader and how you test it
- Require a "diff of understanding": they annotate three AI lines they changed and why
- Ban AI on the midterm if the midterm is about unaided explanation; say so
- Allow AI on a take-home **only** with disclosure and an oral

Do not run a secret "AI detector" as sole evidence. Those tools are unreliable. Use the student's performance on the work.

Plagiarism in writing

Theses and reports: teach citation in week 1 of the project, not after the misconduct meeting. Show a bad paraphrase. Show a good one. Point at 05 Sample Graduation Thesis as a structure, not as text to reuse.

Exercise

Rewrite your integrity paragraph to 120 words. Then add one oral or live-tweak checkpoint to the project so authorship is visible.

13 — Classroom difficulties

Most weeks are ordinary. Some minutes are not. Prepare a few scripts so you do not invent policy under adrenaline.

Silence

“Any questions?” is not a method. See 20 Questioning and Discussion.

If the room is silent after a hard idea:

- Give a 60-second write
- Then pair
- Then take two written answers
- Then you talk

Silence after “clone the repo” usually means the repo is broken.

The student who dominates

Thank them, park it, widen:

“I want to hear that idea in a minute. I am going to take two other voices first.”

Talk to them privately if it continues: you value their energy; the room needs air. Give them a challenge valve or a TA-style job (note the board), not a license to lecture.

Side talk and phones

Proximity first. Stand near. If it continues, a calm public line:

“I need this table on the shader for five minutes.”

Do not humiliate. Do not hold a 10-minute speech about respect; it rewards the disruption.

If a student is on a laptop doing other work, you may set a laptop-down stretch for 8 minutes of board work. Do not run a police state. Design the class so there is something to do with the machine.

Chronic lateness

Start on time with a retrieval quiz. Late arrival then has a cost that is not a scolding. If someone is late every time because of work or transit, problem-solve once privately.

Challenges to your expertise

A student says you are wrong. Sometimes they are right.

“Show me. If you are right, we will all learn it. If not, we will find the mismatch in convention.”

If you were wrong, say so, fix the note, move on. Authority grows.

If it is a performance, timebox: “Office hours. We have a lab to start.”

Anger, insults, harassment

- Stop the content.
- “That language is not allowed in this room.”
- If it is aimed at a person: separate, document, follow Title IX / campus harassment process as required.
- Do not debate slurs.

If you feel unsafe, end the meeting. Call campus security. You are not required to absorb abuse to be a dedicated teacher.

Crying, panic, shutdown

Quiet voice. Offer water and a break. Do not make them explain in front of the room. See 11 Mentoring and Office Hours for crisis limits.

Group explosion

A team emails that one member did nothing. Do not adjudicate by vibe.

- Look at commits, issues, and the contribution statement
- Meet them together once, then separately if needed
- Use the written process in the spec
- Split the grade only if the spec allows it
- Document

Prevent this with milestones and individual orals (09 Assessment Design).

Cheating accusation in public

Do not accuse in the lab. “Please see me after class” and then the process in 12 Academic Integrity and AI.

You lost your temper

Stop. Apologize for the tone, not for a standard. Repair in the next meeting if needed. Tell a mentor if you are ashamed; hiding it makes it a habit.

Recording and privacy

If students record you, follow campus policy. Do not record students without a rule. Do not post a student's broken demo to social media, even as a joke.

Romantic or sexual interest

If you feel it, you do not act on it. Transfer supervision if you must. If a student expresses interest, refuse clearly, document, tell the chair if needed. This is not optional professionalism.

Illness and your absence

Have a backup: a recorded demo, a TA-led lab, a reading + quiz. Do not teach with a fever to prove dedication. The room will remember the quality, not the martyrdom.

Exercise

Write four scripts on a card: dominate, slur, "you're wrong," public integrity. Practice them once out loud. Awkward in your kitchen is better than awkward in front of 40 people.

14 — TAs and course operations

A course is a small organization. If you treat it as a series of talks, the organization will fail in public: late grades, conflicting advice, a broken starter, a TA who invents policy.

What a TA is for

Role	TA does	Instructor still does
Lab help	Desk visits, install, the weekly bug	Design of the lab, stop-the-room teaching
Grading	Rubric application	Rubric design, calibration, appeals
Forum	First-line answers within 24 hours	Policy, edge cases
Demo support	Second machine, camera, clone test	The live-coding narrative
Never	Writing new policy, changing grades alone, dating students they grade	—

Write this on one page. Give it to TAs before week 1.

Hiring and briefing

Prefer TAs who have **done the labs**, not only those with a high exam score. A quiet student who can sit beside someone is better than a brilliant student who grabs the keyboard.

Pre-semester briefing (90 minutes):

- The learning goals and the weekly machine
- The integrity and AI script
- How to do a 4-minute desk visit (07 Labs and Studio)
- What they must escalate (harassment, crisis, cheating, grade fights)
- The comment bank
- Pay, hours, and “do not work 20 unpaid hours”

Calibration

Before each big grade event:

1. Instructor grades two submissions out loud
2. TAs grade the same two
3. Discuss disagreements until the rubric words change or the scores match
4. Only then open the pile

If you skip this, you are running two courses.

Communication channels

Pick few:

- LMS for official deadlines
- Repo issues or a forum for technical help
- Email for personal / confidential
- Chat only if you or a TA will moderate it

A dead Discord is worse than none. A chaotic Discord is a second unpaid job.

Response-time promise: “Technical questions: 24 hours on weekdays. We do not answer homework help after 20:00 the night before a deadline.” Then keep it.

The course repository

From 02 Curriculum Design Advice:

```
course/  
  lecture/  
  starter/  
  solutions/      # private or released after the deadline  
  assignments/  
  resources/  
  admin/          # rubric keys, TA notes – not public
```

Rules:

- `main` always has a running starter
- Secrets and solution keys are not in the student remote
- Tag releases (`week-03-starter`)
- A CONTRIBUTING or TA.md that says how to open a PR for a typo

Weekly operations rhythm

When	Who	What
Friday	Instructor	Next week's starter + lab freeze
Sunday	TA	Run starter on a clean machine
Lecture day	Both	Quiz copies, HDMI, backup demo
Lab day	Both	10-minute pre-brief
48h after lab	TAs	Grades + two comments
After grades	Instructor	10% audit + class-wide note

Records

Keep: syllabus version, emailed exceptions, integrity cases, accommodation letters, gradebook exports.

Do not keep: jokes about students in a shared meme channel.

When a TA is struggling

Coach once with a concrete behavior ("you finished their shader; next time leave them at the checklist"). If they cannot follow integrity or respect rules, remove them from grading. Document. Tell the chair.

When you have no TA

Cut scope. Autograde more. Oral feedback in lab. Shorter projects. You cannot secretly be three people. See 15 Time Management for Instructors.

Exercise

Write a one-page TA contract: hours, roles, escalation, channels, calibration date. If you cannot, the course will train TAs by accident.

15 — Time management for instructors

Teaching expands to fill every evening if you let it. A good teacher is available enough to diagnose and not so available that they never prepare, rest, or do research.

Burnout looks like irritability in lab, late grades, and cynicism about “this generation.” Prevent it as an operations problem.

Count the real hours

A 3-credit course is not 3 hours. A honest week:

Task	Hours (typical)
Contact (lecture + lab)	4–6
Prep (first year)	6–10
Prep (later years, if you reuse)	2–4
Office hours	2
Grading (if no TA, 40 students)	4–8
Email / forum	1–3
Course improvements	1–2

If the department assigned you three new preps, you must cut something: assessment frequency, project ambition, or research. Do it in writing with your chair. Silent heroism fails in week 10.

Prep that scales

- Reuse the visualizer and the kernel
- Change one thing per week, not the whole stack
- Record a demo once; live-code the decisions around it
- A comment bank
- A quiz bank
- Freeze the lab on Friday; do not rewrite it at 8:00

The first year is expensive. Year 2 should be cheaper if you kept notes (03 Lesson Planning).

Grading that scales

- Rubrics with few cells
- Autograde the kernel
- Human-grade the picture and the paragraph
- Return a class-wide pattern in lecture instead of 40 identical essays
- Do not accept “just look at my GitHub” as a submission format

See 08 Feedback and Rubrics.

Email rules

- Batch twice a day
- A FAQ page you can paste
- "If this is a grade dispute, use the regrade form"
- You do not owe a same-hour answer on Sunday

Students will adapt to the system you actually run, not the one you wish you had.

Saying no

You will be asked to: guest-lecture, join a committee, advise extra theses, write a letter this week, add a student after week 3, change a deadline for a whole class because a different course had a midterm.

Use a criterion: does this help **this** course's learning goals this term? If not, "not this semester" is a complete sentence.

Cap thesis load. A careful supervisor with four students beats a neglectful supervisor with twelve. See 22 Supervising Theses.

Calendar hygiene

- Put prep on the calendar as a meeting with yourself
- Put grade-return dates on the student calendar so you are stuck with them
- Protect one half-day with no teaching tasks
- Sleep. A tired live-coder is a rude live-coder

Perfectionism

A B+ lab that exists on Monday beats an A+ lab that appears Wednesday at noon. Ship the starter.

Your lecture does not need to be a conference talk. It needs a goal, a picture, and an attempt.

Life

Caregiving, illness, and research deadlines are real. Tell the chair early. Use the backup plan in 13 Classroom Difficulties. Students can survive a cancelled lecture. They cannot survive a semester of a disappearing examiner.

Exercise

Track one week in 30-minute blocks. Circle the hours that did not help a student learn or help you recover. Cut one of those recurring blocks next week.

16 — Teaching portfolio and growth

You will not become good by intending to. You become good by looking at evidence and changing one practice.

A teaching portfolio is that evidence, arranged so a hiring committee, a promotion file, or your future self can see it.

What to keep (from week 1)

- Syllabus versions (date them)
- Two sample assignments + rubrics + anonymized student work (with permission or campus policy)
- A weekly log: what failed, what you changed (01 What Good Teaching Looks Like)
- Student evaluations **and** your written response (what you accept, what you reject, what you will change)
- A peer observation (someone sits in; you sit in on them)
- One recording of live coding
- Project showcase links (class, not individual grades)

Do not keep only the glowing emails.

A simple portfolio structure

1. **Teaching statement** (2 pages): what you think learning is, how you teach IGWT, one example that went wrong and what you changed
2. **Course list** with your role
3. **Representative materials** (one course designed end-to-end is stronger than seven slide decks)
4. **Evidence of student learning** (before/after quiz, project run rates, not only smiles)
5. **Improvement loop** (year 1 vs year 2 of the same course)
6. **Service to teaching** (TA training, curriculum, 01 subjects work)

Committees can smell a statement made of slogans. Use a concrete scene: the week the starter broke, the week you added a planted bug, the week orals replaced a take-home.

Observing and being observed

Ask a colleague to watch 20 minutes with a form:

Question	Notes
What was the learning goal?	
Who spoke?	
When did students do something?	
Was the demo visible?	
What would you try next time?	

Then you watch them. Mutual observation is less threatening and more useful than a single "evaluation visit."

Student evaluations (how to read them)

They measure a mix of learning, entertainment, gender and accent bias, and whether the projector worked.

Use: recurring comments about pace, clarity, feedback speed, respect.

Do not use as sole truth: a single cruel paragraph; a score without comments; "more entertaining."

Write a 200-word response each term: three changes you will make, one thing you will not change and why (for example, you will keep weekly quizzes).

Scholarship of teaching

You do not need a journal paper. You can:

- A/B a starter (with ethics approval if required)
- Count how many pairs pass the hidden test after you change the lecture
- Present a teaching talk in the department

Treat teaching questions like engineering questions: hypothesis, change, evidence.

Career paths

- **New instructor:** survive with 23 First Year as an Instructor
- **Job market:** one designed course + portfolio site + a talk that is not only research
- **Promotion:** year-over-year improvement and leadership (TAs, curriculum)
- **Industry-to-academia:** your advantage is live practice; your gap is assessment and inclusion — study those notes

Rest is part of growth

A portfolio built on 70-hour weeks is a warning label. See 15 Time Management for Instructors.

Exercise

Write a 400-word teaching statement that includes one failure. If you cannot name a failure, you are not looking yet.

17 — Common mistakes

These are the failure modes that look like hard work from the inside.

Covering instead of teaching

You “got through” clip space, PBR, and shadows. Students can do none of them. Coverage is a plan. Learning is a result.

Repair: one idea, one attempt, one check. Cut the third topic.

Slides as the course

The deck is a 90-page PDF. Lab is “do the tutorial.” You are a projector operator.

Repair: board the argument; lab continues the same object; slides hold pictures.

Demo of a finished cathedral

You open a production repo and click play. Students are impressed and learn nothing they can redo.

Repair: start from their starter. Build one slice. See 06 Live Coding Pedagogy.

Environment as the exam

Half the lab is NVIDIA drivers, permissions, and `npm`. The geometric idea never appears.

Repair: one-command starter, lab image, pre-lab install hour.

Invisible success condition

“Make something cool with Three.js.” Anxiety, then three.js examples pasted.

Repair: a screenshot, a test, a rubric. Cool is the “exceeds” row.

Feedback after the course is over

Final comments on a project they will never open again.

Repair: milestones and a week to act. 08 Feedback and Rubrics

Grading theater

You spend 12 minutes per lab writing essays. You are late. Students do not read them.

Repair: rubric + two comments + class-wide pattern.

Moving the stack every year

WebGL one year, WebGPU the next, a new framework in week 3 because you are bored.

Repair: freeze the student-facing stack for the term. You may preview new tech in week 15.

Teaching only the students who are already good

You riff with the front row. The back row is lost and polite.

Repair: look at the starter files, not the faces. Target the middle. Valves for the top.

Humiliation as standards

Sarcasm, surprise orals for people who fear them, public shame for a black screen.

Repair: high standards, private correction, public principles. 10 Inclusive Teaching and Accessibility

Policy you will not keep

"No late work." Then you accept everything. Students learn that written rules are fiction.

Repair: write the rule you will actually run.

Ignoring AI

A take-home essay is now a weak assessment.

Repair: 12 Academic Integrity and AI

Scope creep in projects

Every idea is "a small engine." Nobody ships.

Repair: must/should/may. Feature freeze. 09 Assessment Design

No names, no presence

You do not know who is drowning.

Repair: roster, lab circulation, one early email. 11 Mentoring and Office Hours

Heroics

You answer Slack at 01:00, rewrite the lab at 08:00, and resent the room at 09:00.

Repair: 15 Time Management for Instructors

Confusing research talks with undergraduate lectures

A seminar on your paper is not Week 3 of Computer Graphics I.

Repair: prerequisites, pictures, an attempt. Save the paper for a guest hour with scaffolding.

Never watching yourself

You have taught five years and have one year of experience five times.

Repair: record, observe, change one thing. 16 Teaching Portfolio and Growth

Exercise

Circle the three mistakes you have made. Write one repair you will run in the next two weeks. Tell a colleague so it is real.

18 — Course-by-course teaching notes

Curriculum map: 01 subjects. Weekly machine: 02 Curriculum Design Advice. This note is the **instructor's field guide**: what is hard, what to freeze, what to assess.

1. Introduction to Programming

Hard: students think typing is thinking; they cannot debug.

Freeze: one language all term.

Do: lots of tiny programs with visible output; a style guide; autograder for functions.

Do not: start with a framework.

Assess: functions, loops, a small project they can explain line by line.

Live coding: errors on purpose. Read the stack trace out loud.

2. Web Technologies

Hard: the browser is an operating system; they only see tags.

Freeze: HTML/CSS/JS without a bundler for weeks.

Do: DevTools as a lab instrument; one layout they clone badly on purpose.

Do not: a full React app in week 2.

Assess: a page they can inspect; explain reflow vs paint at a basic level.

3. Mathematics for Computer Graphics

Hard: math class energy; they do not see a camera.

Freeze: 2D first, then 3D; one convention for column/row.

Do: every week a picture and a tiny code check (even in Python).

Do not: proofs without a geometric object.

Assess: transform a point; interpret a dot product; a short oral with a whiteboard.

Computational Geometry (Semester 2)

Full plan: 04 Computational Geometry. Notes: 00 Lectures.

Hard: degeneracy and floating point; they want libraries.

Freeze: `orient` sign and the shared visualizer.

Do: hidden fixtures; plant collinear points.

Do not: 3D computational geometry in week 4.

Assess: kernel tests + visual + a short invariant question.

4. Computer Graphics I

Hard: spaces (object/world/view/clip); they mix them.

Freeze: right-handed, one engine (Three.js **or** a tiny custom pipeline, not both at once).

Do: move the camera live; draw the spaces every lecture.

Do not: PBR in week 2.

Assess: a scene with a camera they can explain; a paper transform.

5. Modern JavaScript Development

Hard: tooling as identity; they configure instead of thinking.

Freeze: one bundler or none; TypeScript only if you will grade types.

Do: modules, async, performance of a real loop (particles, not TODO apps only).

Assess: a small library with tests; a performance note.

6. Interactive Web Development

Hard: animation as decoration; they cannot reason about time.

Do: requestAnimationFrame, easing, one GSAP sequence they rebuild.

Do not: five libraries.

Assess: a motion they can retarget; accessibility of motion (prefers-reduced-motion).

7. WebGL Programming

Hard: state machine + shaders + buffers at once.

Freeze: raw WebGL (or a 50-line wrapper you wrote). No Three.js this course.

Do: triangle → square → cube → texture → light. Draw the pipeline every time.

Do not: start with a 1,000-line engine.

Assess: they can get a black screen to a triangle using the checklist (06 Live Coding Pedagogy).

8. Three.js Development

Hard: the library hides the pipeline they just learned; they forget it.

Do: keep asking “what is this in GL?” Load glTF; lights; a simple optimize (instance or fewer lights).

Do not: every add-on in the examples folder.

Assess: a loaded model, one interaction, a README that runs.

9. Blender for Real-Time Graphics

Hard: film habits (too many polys, huge textures, wrong scale).

Freeze: glTF export path; meters; texel density target.

Do: budget sheet (tris, maps, materials). Look at the model in the Three.js viewer the same day.

Assess: an asset that runs at a stated budget, not a beautiful sculpture that dies on mobile.

10. Shader Programming

Hard: they copy Book of Shaders aesthetics without a model of the pipeline.

Do: one change → one picture. Noise, then a use (terrain, fire). Warn for flashing.

Assess: five small shaders with comments on the math, or two deeper ones. Oral: "what is in the fragment?"

11. Real-Time Rendering

Hard: vocabulary explosion (PBR, IBL, AO, HDR).

Do: one effect per week, with a before/after and a cost.

Do not: implement every paper.

Assess: a scene with two effects they can turn off and measure.

12. GPU Programming

Hard: GPGPU mental model; they think the CPU loop is the same.

Do: particles or a small sim; then a WebGPU preview without abandoning the term's stack.

Assess: a measurement and a limitation paragraph.

13. Interactive Experience Development

Hard: design taste + engineering; they hide in one.

Do: R3F (or equivalent) as architecture, not magic. Scroll, UI, audio as systems.

Assess: a small award-style slice that is finishable; critique on criteria (07 Labs and Studio).

14. Virtual & Augmented Reality

Hard: hardware inequality; comfort (locomotion, IPD, nausea).

Do: interaction first, headset second. Provide lab headsets. Comfort checklist.

Assess: an interaction that works seated if needed; a safety/comfort note.

15. AI for Interactive Graphics

Hard: they generate assets they cannot control; ethics is a slide they skip.

Do: AI as a pipeline stage with licenses, disclosure, and a human-editable output. See 12 Academic Integrity and AI.

Assess: a system that uses an API **and** a critique of failure modes, not a folder of pretty images.

16. Advanced Computer Graphics

Hard: papers. Undergraduates drown.

Do: one paper carefully; implement a toy version; compare to the production approximation.

Assess: a reproduction attempt with honesty about what they did not implement.

17. Capstone

Hard: teams, scope, denial.

Do: sprints, reviews, external demo, individual orals. Graduation Requirements

Do not: a new research field in 15 weeks.

Assess: running system + docs + talk + individual understanding.

Exercise

For the course you teach next, write the **Hard / Freeze / Do / Do not / Assess** five lines on a card. If you cannot, you are still covering, not teaching.

19 — Checklists and templates

Print these. Fill them. Do not admire them.

A. Week-before-class

- ☐ Learning goal is one observable sentence
- ☐ Starter runs on a clean clone
- ☐ Solution exists in a private place
- ☐ Lab has a screenshot of “done”
- ☐ Quiz: 5 items + key
- ☐ Demo backup (recording or known-good commit)
- ☐ Rubric matches the spec
- ☐ TAs briefed on the week’s misconception
- ☐ Accommodation list checked (quietly)
- ☐ HDMI / audio / markers work

B. Day-of lecture

- ☐ Room 10 minutes early
- ☐ Editor zoom 140%+
- ☐ Opening retrieval ready
- ☐ Board plan: goal + invariant strip
- ☐ Student attempt exists in the middle
- ☐ Ending: lab hook, not “any questions?”

C. Lab open

- ☐ Success condition said in one sentence
- ☐ Expected bug named
- ☐ Pair roles if pairing
- ☐ Queue rule for help
- ☐ You will circulate, not sit

D. Desk visit (4 minutes)

- ☐ “Show me what it does”
- ☐ “What did you try?”
- ☐ One next action
- ☐ Leave

E. Help request (students)

Expected:
Got:
Tried (2):
Hypothesis:

F. One-page lesson plan

See 03 Lesson Planning for the full template. Minimum:

Goal:
Check:
0-10:
10-25:
25-40:
40-55:
55-75:
If demo dies:
Cut:

G. Assignment spec (minimum)

Due:
Must:
Should:
May:
Forbidden:
Deliverables:
Rubric link:
Integrity / AI (4 lines):
How to get help:

H. Rubric (copy and edit)

Criterion	Below	Meets	Exceeds
Runs from README			
Core technique			
Visible idea			
Code / repo			
Write-up			

I. Project milestones

Week 2: repo + topic
Week 4: core feature runs
Week 6: limitation note
Week 8: freeze + draft report
Last week: demo

J. Integrity four-liner

Discuss ideas: yes.
Copy kernels / reports: no.
AI with disclosure and understanding: yes.
AI as invisible author: no.

K. Black-screen checklist

1. Canvas / size
2. Clear color visible
3. Shader compile log
4. Camera vs object
5. Near plane
6. Winding / cull
7. Depth
8. Attributes
9. Async texture
10. Color / alpha

L. After-class log (4 lines)

They could:
They stuck:
I will change:
I will check on:

M. Office hours door card

Hours:
Queue: name on board, 12 minutes
Bring: repo + the help template
Emergencies: campus counseling / security numbers

N. TA escalation

Escalate immediately: harassment, safety, cheating case, medical crisis, "the starter is broken for everyone."

Do not escalate: one student forgot `npm install` (unless it is everyone).

O. First-day tiny win

Pick one from 04 First Day and Syllabus and write the exact command they will run.

P. Observation form

From 16 Teaching Portfolio and Growth: goal, who spoke, student action, demo visibility, one suggestion.

Q. Regrade request

Assignment:
Rubric cell:
Evidence in my submission:
What I think the score should be:

R. Thesis meeting (15 minutes)

What runs:
Risk:
Cut:
Next artifact (date):

Exercise

Print A, D, J, and K this week. Use them once. Then they exist.

20 — Questioning and discussion

The quality of the class is the quality of the questions and the wait after them. A lecture full of rhetorical “right?” is a monologue with extra syllables.

Wait time

Ask. Then be quiet for **5–7 seconds**. It will feel like a minute. If you answer your own question at second two, you have trained them not to think.

If the room is large, take written answers first so thinking is not public yet.

A ladder of questions

Level	Example in IGWT	When
Recall	“What does <code>orient > 0</code> mean in our convention?”	Opening retrieval
Describe	“What is on the screen after the vertex shader?”	After a demo
Predict	“If I flip the winding, what happens?”	Before you flip it
Diagnose	“The mesh is black. Where do you look first?”	Checklist practice
Justify	“Why not sort in place here?”	After they have a kernel
Transfer	“Where does this show up in picking?”	End of a topic
Critique	“Does this README let a stranger run the project?”	Studio

Do not start a new topic with a critique question. Novices cannot critique a schema they do not have.

Cold call, warm call

Warm call: warn them (“I will ask you to trace this in a minute”). Humane and effective.

Cold call: use sparingly, after norms exist, never as punishment.

Never: pick on someone who looks lost to make an example of them.

A good pattern: “Write for 45 seconds. Pair. I will ask two pairs.” Then it is not a raid.

Wrong answers

Wrong answers are data.

“That would be true if we were in view space. We are still in world space. How would we check?”

Do not say “no” and move to a favorite student. The room learns that only finished thoughts are welcome.

If the answer is unsafe (harassment), that is 13 Classroom Difficulties, not a Socratic moment.

Pair and write

Think-pair-share is not a gimmick. It is how you get more than three voices.

- 45–90 seconds silent write
- 2 minutes pair
- Harvest 2–3 answers on the board (names optional)

Use a shared doc in large rooms if speaking is a barrier (10 Inclusive Teaching and Accessibility).

Discussion that is not a bull session

Give a **text or artifact**: a 15-line kernel, a screenshot, a short paragraph from a paper, a README.

Questions:

1. What is the claim?
2. What would falsify it?
3. What would you change?

Timebox. End with a written takeaway so it does not evaporate.

Handling “I don’t know”

Praise the honesty. Narrow:

“Is the problem the sign, the sort, or the visualizer?”

Give a 50/50 if needed. Then they can enter.

Online / hybrid

Do not ask “questions in chat” and ignore the chat. Assign a TA as chat navigator. Repeat the question aloud for the recording.

Exercise

Write six questions for your next lecture, one from each ladder row except critique. Put the predict question **before** the demo, not after.

21 — Writing assignments

A homework is a designed practice, not a pile of leftover slides. Bad homework is either trivial busywork or a second unpublished course.

Properties of a good weekly assignment

1. **Continues the lab object** when possible (same repo, next function)
2. **Has a must layer** that an average prepared student can finish in the advertised hours
3. **Has a stretch** that is clearly optional
4. **Mixes doing and explaining** (4–6 hours code + 3 short written items is a good default)
5. **Is testable or screenshotable**
6. **Does not require unpublished lecture jokes**
7. **Names the integrity rule again** (one line)

Advertised hours must be real. If you would need 12 hours, you advertised wrong or the starter is wrong.

Written questions that work

- A picture to mark
- "Here is a wrong student explanation. Fix it."
- "Give a degenerate input and what your program does"
- "Complexity of step 2 after the sort, and why"
- "Paste a 10-line function and comment the invariant"

Avoid:

- "Research Three.js and write 800 words"
- "Read the chapter and summarize"
- Questions whose only answer is a definition you will never use again

Code questions that work

- Implement `onSegment` given `orient`
- Add a fixture that currently fails
- Change one constraint ("no extra allocations in the inner loop" or "handle duplicates")
- A refactor: extract the kernel from the draw loop

Avoid:

- "Build a game"
- "Match this screenshot exactly" with an underspecified camera
- Hidden dependencies on a TA's machine path

Starters

A starter should compile, contain `TODO` s, and include **one failing test** or a clearly wrong picture. An empty folder is not a starter. A finished solution with `TODO` comments is a cruelty.

Homework vs lab

Lab: first contact, pairs allowed if you say so, help in the room.

Homework: they should be able to finish the must layer alone. If they cannot, the lab did not end at the right place.

Late days

A small bank of late days (for example 3×24 hours per term) reduces email and panic. State whether they apply to projects. Do not combine mystery extensions with a public “no late work” line.

Returning

Comments before the next related work. A key or a short walkthrough after the deadline, or the homework is a black box.

Multi-week projects

See 09 Assessment Design for milestones. The weekly homework in project season should be **artifacts** (risk list, draft figure, failing test), not new feature lists.

Reading assignments

If you assign a paper or a chapter:

- Give 3 questions they must answer
- Tell them what to skip
- Use 10 minutes of class on the questions, or they will not read

A 40-page paper with “please read” is a wish.

Exercise

Write next homework in the advertised hours yourself, from the student starter, without your extra aliases. If you cannot, they cannot.

22 — Supervising theses

Graduation standards: Graduation Requirements. Short model: 05 Sample Graduation Thesis. Long archive: 03 Reference Thesis. This note is the **supervisor's** job, not the student's outline.

A thesis in IGWT is project-based: a running system plus a written argument. If you supervise only the vibes of the idea, you will meet a beautiful slide deck and a repo that does not clone.

How many students

Be explicit with yourself. A serious undergraduate thesis is about 1 hour every 1–2 weeks plus extra near the end, if they are on track; more if they are not. Four students you see is better than ten you forget.

If the department assigns you too many, ask for co-supervisors or a studio model with shared meetings.

Choosing a topic

Good undergraduate topics are **narrow, visible, and evaluable**.

Good	Dangerous
Hull + Delaunay + picking in a browser, evaluated on fixtures	"A complete metaverse platform"
One PBR feature compared with a baseline	"A new global illumination algorithm"
Configurator for 3 parts with measured load time	"AI-generated everything"
WebXR seated exhibit with a comfort study	"Full locomotion MMO"

The sample thesis exists so you can say: "This length, this honesty about timing tables."

Force a sentence:

I will build X so that a user can Y; I will know it worked when Z.

If Z is "it looks cool," it is not a thesis yet.

The supervision contract (week 0–1)

Write one page:

- Meeting rhythm (every 2 weeks, 20 minutes, they send an agenda)
- What "on track" means each month
- Tools and AI disclosure
- Authorship: they write; you do not ghostwrite
- What you will read (drafts with a date, not a dump the night before)

- How they reach you in a real emergency
- Exhibition and defense dates from Graduation Requirements

They run the meeting. You do not hunt them for news.

A 15-week supervision map (final semester)

Align with the table in Graduation Requirements.

Weeks	Supervisor looks at	Kill / cut if
1–2	Proposal: X/Y/Z sentence, related work list (10 sources), repo	No sentence, no repo
3–6	Design figure, kernel or scene running, ethics/AI note	Still only mood boards
7–11	Feature freeze approaching, first measurements or a reason there are none	New scope every meeting
12	Evaluation chapter: real numbers or labeled templates	Invented FPS
13	Thesis PDF compiles, citations exist	First complete draft
14	Exhibition rehearsal	Demo only on their machine
15	Defense questions (give them a list)	They cannot explain the kernel

Give the Week 14 defense-style questions idea from Week 14 Project Studio even if the course is not computational geometry.

Reading drafts

You are not a copy editor for 80 pages every weekend.

- First draft: structure and claims, not commas
- Second: figures, evaluation honesty, related work coverage
- Third: only what a second examiner will attack

Use the comment shape in 08 Feedback and Rubrics. Demand captions. Demand that every performance number say how it was measured — or that it is a template, as this repository's samples do.

Do not write their chapters. Line-editing a weak argument into your voice is ghostwriting. Ask questions that force them to rewrite.

Evaluation and honesty

This program's sample documents warn that timing tables may be templates. Students must not submit templates as measurements. Your job is to ask:

- What machine?
- What n?
- How many runs?
- What did you change between conditions?

If they did not measure, the thesis must say so. A failed experiment is acceptable. A fake table is not. See 12 Academic Integrity and AI.

Group capstones vs individual theses

If the software is shared, the **written argument and the oral** must still show an individual mind. Split chapters by ownership. Ask each person to change a part live.

Committee and defense

Brief the student on the format in Graduation Requirements: 15–20 minutes, live demo, questions.

Give them ten questions in advance; ask two of them. Sample:

1. What is the core technical claim?
2. What would falsify it?
3. Show a failure case.
4. Why this algorithm / architecture and not the obvious alternative?
5. What did you measure, and what did you not?
6. What would break on a mid-range phone?
7. Where did AI or other code enter, and what can you change without it?
8. What would you cut if you had four weeks left?
9. What is the next experiment, exactly?
10. What should a second-year student steal from your repo?

If the demo dies, they should have a video. That is professionalism, not cheating.

When they disappear

Email once with a date. Email again with the chair copied if policy allows. Document. You cannot supervise a ghost, and you must not invent a passing thesis.

When the work is not passing

Say so in week 6, not at the defense. Offer: cut to a smaller X, delay if the university allows, or fail with a path to retry. Kindness that hides a fail until week 15 is cruelty.

Letters and authorship

If the work becomes a paper, discuss authorship early. Undergraduates who did the implementation belong on the author list if they meet the field's standard. Do not surprise them.

Exercise

Write the one-page contract for your next thesis student. If you already have students and no contract, write it this week and send it.

23 — First year as an instructor

You do not need to become a legendary teacher by December. You need to run a fair course, learn names, keep the starter running, and change three things in year 2.

This note is a calendar, not a pep talk.

Thirty days before term

- Read 00 How to Use These Notes and the 90-minute path
- Write the 1-page syllabus (04 First Day and Syllabus)
- Freeze the student stack (language, repo, one graphics API)
- Build week 1–3 starters until a friend can clone them
- Write the integrity four-liner (12 Academic Integrity and AI)
- Find the room, the projector, the lab image, the disability office process
- Ask who your TAs are; if none, cut assessment load now (15 Time Management for Instructors)
- Schedule office hours into your calendar as if they were a class
- Ask one experienced instructor: “What surprised you in week 3?”

Do not spend the month redesigning the entire six-semester program. That is 01 subjects work for a later year.

Week 0 (the week before students)

- Print checklists A, J, K from 19 Checklists and Templates
- Record a 4-minute backup of the first demo
- Send the five-line email from 04 First Day and Syllabus
- Sleep.

Weeks 1–3

Goals: names, a tiny win, a diagnostic, a working help system.

- Do not add topics because you are nervous
- Circulate in every lab
- Log four lines after each week
- Return the first graded thing fast, even if it is a tiny quiz
- Invite the quiet students who failed the diagnostic

If week 1 is an install disaster, that is your emergency, not theirs. Fix the starter; do not blame the cohort.

Weeks 4–7

The novelty is gone. This is where courses die.

- Keep quizzes short
- Stop-the-room when three desks share a bug
- Calibrate TAs before a big grade
- Watch your email hours
- Plan the project spec **now**, not in week 10

Midterm: pictures and traces, not trivia. 09 Assessment Design

Weeks 8–11

Project season. Your job shifts from new content to **diagnosis and cuts**.

- Feature freeze date in writing
- Individual checkpoints so groups cannot hide
- One studio critique with criteria
- Do not start a new framework because you are bored

Weeks 12–15

- Exhibition and defense logistics (Graduation Requirements, 22 Supervising Theses)
- Regrade window
- Student evaluations: ask for specific comments on starter, feedback speed, respect
- Write the year-1 vs year-2 note while you still remember

Do not assign a new epic in week 13.

What “good enough” looks like in year 1

- Students can do the must-layer labs
- Grades are explainable from a rubric
- No safety or harassment case was ignored
- You know who was drowning and you contacted them
- The repo clones
- You have a list of five changes for next year

You will not love every lecture. That is normal.

What to refuse in year 1

- A third new prep
- Twelve thesis students
- A complete rewrite of the stack in week 5

- Unpaid weekend Discord
- Committees that are not required

Your chair would rather you teach one course well.

Find people

- A teaching mentor (not only a research advisor)
- A peer to observe (16 Teaching Portfolio and Growth)
- A TA you can trust to tell you the starter is bad

Do not isolate. Isolation turns ordinary problems into shame.

If you came from industry

Your advantage: you have seen production pain; you can live-code for real.

Your gaps: assessment design, accommodations, academic integrity process, the pace of novices.

Read 08 Feedback and Rubrics, 10 Inclusive Teaching and Accessibility, 12 Academic Integrity and AI twice.

Do not treat students like junior employees who should already know Git. Teaching Git is part of the job.

If you came from a PhD

Your advantage: you can explain invariants.

Your gap: you may lecture like a seminar and never ship a starter.

Force yourself to do 06 Live Coding Pedagogy every week. Watch the clock. Cut the paper.

End-of-year packet (2 hours)

1. Five changes
2. Two assignments you will keep
3. Anonymized examples of strong / weak work
4. Your four-line logs, skimmed
5. A thank-you to TAs with specific praise

That packet is the beginning of a portfolio.

Exercise

Write your 30-day-before list with actual dates. If term starts in less than 30 days, do the first five bullets tomorrow.